多玩我的世界盒子加密资源分析与恢复研究报告

生成日期：2026-09-03

研究对象：多玩我的世界盒子 2.6.2 APK 反编译源码、用户自有付费地图与 modpkg 样本

研究目的：在多玩服务器停止服务后，对已购买资源进行离线解密、修复与个人归档，恢复其在 MCPE 中的正常加载能力。

1. 范围与合规说明

本报告只讨论对用户已经合法拥有的多玩资源进行离线恢复的技术路径。报告中的结论来自本地 APK 反编译代码、native so 调用行为、样本资源结构、解密工具运行日志以及 MCPE 实机验证。本文不讨论伪造购买状态、绕过账号校验以使用未拥有资源、重新分发付费资源等用途。

2. 总体结论

多玩付费资源的保护不是单一算法，而是“文件级加密 + key 元数据 + 本地资源数据库状态 + MCPE 悬浮窗插件 hook + LevelDB 注入”的组合。

对于资源内容本身，多玩主要使用 `liblauncherutil.so` 暴露给 Java 层的 native 方法完成加密和解密。反编译代码中可见的关键入口包括：

- `LauncherMiscUtil.decryptFileData(byte[])`
- `LauncherMiscUtil.decryptFileDataUsekey(byte[], byte[])`
- `LauncherMiscUtil.encryptFileData(byte[])`
- `LauncherMiscUtil.encryptMapData(byte[], byte[])`
- `LauncherMiscUtil.generateAesKey()`
- `LauncherMiscUtil.decryptModule(byte[], int)`

资源文件层面常见两种加密形态：

- 带 `19311931EE` 魔数的头部封装文件。
- 以 `数字#密文` 或多个片段用 `@` 拼接的文本式加密内容，常见于地图、材质包、行为包中的 JSON。

地图资源还存在额外的 LevelDB 修复问题。122B 的 `.log` 文件不是普通日志，而是 LevelDB WAL 记录，写入键 `mVillages`，其值形如 `64#...`。MCPE 打开世界数据库时会重放 WAL，导致该值合并进 `.ldb` 并使原 `.log` 变成 0B 或被轮转。若不处理，原版 MCPE 或多玩盒子加载地图时可能崩溃或提示地图损坏。

最终实机验证表明：在正确解密资源文件、删除多玩 key 元数据、修复 LevelDB `mVillages` 注入、补齐 LevelDB 新生成的 `MANIFEST-xxxxx` 文件后，原版 MCPE 与多玩导入均可以加载新解密地图，材质包和行为包逻辑也能正常工作。

3. 关键代码证据

3.1 native 加解密接口

项目中复原出的 Java 壳类位于：

`app/src/main/java/com/mcbox/pesdk/util/LauncherMiscUtil.java`

该类加载 `liblauncherutil.so`，并声明多个 native 方法。实际算法主体在 so 内部，Java 层主要负责传入待解密字节、key 或模块类型。

当前解密工具通过这些 native 方法复用原 APK 的解密逻辑，避免重新实现未知的对称算法细节。

3.2 付费地图 key 生成与校验

反编译代码中，下载付费地图时会创建 `.level.key` 与 `.user.key`。其中 `.user.key` 记录用户账号信息以及 `.level.key` 的 MD5，用于本地校验资源是否属于当前账号或是否被篡改。

相关路径：

? `smali/com/mcbox/app/task/ResourceDownloadTask.smali`

? `smali/com/mcbox/app/task/h.smali`

本地扫描地图时，多玩会读取 `.user.key`，调用 `LauncherMiscUtil.decryptFileData` 解密后提取 `userId`，并结合本地资源数据库判断是否需要登录购买账号。

3.3 多玩盒子资源损坏与付费资源提示

反编译 Java 中可见多处资源状态判断：

? `sources/com/duowan/groundhog/mctools/activity/map/be.java`

- 读取 `getEncryptType()`，若大于 0 则显示加密资源提示。

? `sources/com/duowan/groundhog/mctools/activity/b/bn.java`

- 使用 `EncryptUtil.isEcrypteFile(file, 2)` 判断当前资源是否加密。
- 显示“以下资源需要登录购买该资源的账号才能加载”。

? `sources/com/duowan/groundhog/mctools/activity/myresource/bh.java`

- 对加密皮肤资源执行 MD5 校验，失败后提示资源已损坏。

? `resources/res/values/strings.xml`

- `dialog_encryresource_pay_tip`
- `resource_damage`

这说明多玩盒子本身会根据资源数据库、key 文件、MD5 和 encryptType 判断资源状态。因此，已经去 DRM 元数据的地图不一定适合继续用多玩盒子的资源列表作为唯一验证目标。最终验证应以原版 MCPE 能否正常导入并加载为主。

3.4 悬浮窗插件与运行时解密

多玩正常打开付费地图时要求启用悬浮窗。代码中 `GameUtils.startMcWithFloatWindow` 会设置多个启动参数，例如用户 ID、付费材质包标志、加密插件状态等。

多玩插件包 `lp_v*.dat` 或运行时释放出的 `lp_v*.apk` 中存在与 MCPE hook 相关的符号和字符串，例如：

- ? `selectLevelHook`
- ? `decodeResource`
- ? `decodeStream`
- ? `decodeUserkeyContent`
- ? `decrypt_map`
- ? `mcpelauncher`

native 层还可见类似：

- ? `bl_Minecraft_selectLevel_hook`
- ? `open`
- ? `read`
- ? `close`

这表明悬浮窗并不只是 UI 功能，而是参与 MCPE 进程中文件读取或世界选择流程的运行时解密和 DRM 检查。

4. 加密文件格式分析

4.1 头部封装格式

多玩加密文件常见开头为 ASCII 魔数：

```
19311931EE
```

当前工具按如下结构解析：

```
magic:      10 bytes, ASCII "19311931EE"
headerLen:  2 bytes, big-endian
header:     headerLen bytes, encrypted header
body:       remaining bytes
```

解密流程：

1. 读取 magic 与 header 长度。
2. 对加密 header 调用 `LauncherMiscUtil.decryptFileData`。
3. 如果 header 长度足够且特定位表示存在内容 key，则取 header 中的 16 字节 key。
4. 对 body 调用 `LauncherMiscUtil.decryptFileDataUsekey`。

5. 如果不存在内容 key，则对 body 调用 `LauncherMiscUtil.decryptFileData`。

该结构可用于 `.level.key`、`.blur`、`.texture.key`、部分图片与普通文件。

4.2 嵌入式图片加密

部分图片不是从文件开头出现 magic，而是在固定偏移处出现 magic。当前样本中观察到的偏移为：

```
0x546
```

工具保留偏移前原始图片头部，对偏移后的加密载荷按头部封装格式解密，再拼回输出。该模式常见于 4D 皮肤或带预览头的图片类资源。

4.3 文本片段式加密

地图、材质包、行为包内的部分 JSON 文件以 `-mc`、`.json-mc`、`pack_manifest-mc.json` 等形式存在。其内容常见格式为：

```
数字#密文
```

有些文件被 `@` 分割成多个密文片段：

```
数字#密文@数字#密文@数字#密文
```

解密流程：

1. 根据当前目录或 ZIP 内路径寻找 `.blur`、`.level.key`、`.texture.key`。
2. 从 key 文件头部解出 16 字节内容 key。
3. 对每个片段调用 `LauncherMiscUtil.decryptFileDataUsekey`。
4. 若按 key 解密失败，再尝试无 key 的 `LauncherMiscUtil.decryptFileData` 作为兼容路径。
5. 输出时去掉 `-mc` 或 `.json-mc` 后缀，恢复为普通 JSON 文件。

5. 不同资源类型的处理策略

5.1 modpkg

modpkg 的解密相对直接。样本中未发现单独 key 文件，但可通过 ZIP 容器或文件内头部封装识别并调用 native 解密。实际测试中，modpkg 是最先成功恢复的资源类型。

推断原因：

? modpkg 更依赖全局 native 解密逻辑或模块解密逻辑。

? 不像地图那样依赖 LevelDB 修复。

? 不一定需要 `.level.key` 或 `.blur` 作为外部 key。

5.2 地图 ZIP / mcworld

地图是最复杂的类型，通常包含：

- ? 世界目录
- ? `level.dat`
- ? `levelname.txt`
- ? `db/`
- ? `world_resource_packs.json`
- ? `world_behavior_packs.json`
- ? `resource_packs/`
- ? `behavior_packs/`
- ? `.level.key`
- ? `.user.key`
- ? 可能存在 `.blur`

解密处理包括：

1. 解析 ZIP 条目，兼容 UTF-8 与 GBK 文件名。
2. 收集 ZIP 内所有 key 文件。
3. 按最长路径前缀匹配 key，处理地图内不同资源包可能使用不同 key 的情况。
4. 解密 `-mc`、`.json-mc`、头部封装文件和嵌入式图片。
5. 跳过 `.level.key`、`.user.key`、`.blur`、`.texture.key` 等多玩元数据。
6. 修复 LevelDB 122B WAL 注入。
7. 修复已经合并进 `.ldb` 的 `mVillages` 加密值。
8. 将 LevelDB 修复后新生成的 `MANIFEST-xxxxx`、`.ldb`、`CURRENT` 等文件完整写回 ZIP。

曾经出现的关键问题是：`db/CURRENT` 指向新生成的 `MANIFEST-003862`，但输出 ZIP
中缺少该文件，导致 MCPE 打开 LevelDB 失败。补齐新生成 DB 文件后，地图能够正常导入并加载。

5.3 材质包

材质包通常包含：

- ? `pack_manifest.json` 或 `manifest.json`
- ? `textures/`
- ? `blocks.json`
- ? `items_client.json`
- ? `sounds.json`
- ? `pack_icon.png`
- ? `.blur` 或 `.texture.key`

其加密重点在 JSON、图片和资源清单文件。解密时需要：

1. 识别并读取 `.blur` 或 `.texture.key`。
2. 解密 `pack_manifest-mc.json`、`manifest-mc.json`、`.json-mc` 等文件。

3. 解密带 magic 的图片或偏移 magic 图片。
4. 删除 key 元数据。
5. 保持资源包目录结构不被破坏。

早期工具虽然无报错，但 MCPE 崩溃，后来确认原因之一是资源包目录结构和 LevelDB 文件不完整。修复后材质包可以随地图正常加载。

5.4 行为包 / addon

行为包通常包含：

- ? `entities/`
- ? `items.json`
- ? `loot_tables/`
- ? `entities.list`
- ? `pack_manifest.json`
- ? `pack_icon.png`
- ? `.blur`

行为包的主要风险是 JSON 未解密或清单路径错误。MCPE 对行为包解析更敏感，错误 JSON、空文件、错误目录层级都可能导致加载失败甚至崩溃。

当前工具通过路径 key 匹配和 `@`
分片解密恢复行为包内容。实机验证中，行为包逻辑已经可以正常加载。

5.5 4D 皮肤与图片资源

4D 皮肤、图片类资源可能使用：

- ? 文件头部 magic 加密。
- ? 固定偏移 `0x546` 后出现 magic 的嵌入式加密。
- ? `.texture.key` 作为 key 元数据。

处理策略是优先检查文件开头 magic，再检查固定偏移 magic。若存在偏移 magic，则保留偏移前字节，仅解密后半段载荷。

6. LevelDB 与 122B log 修复分析

6.1 122B log 的结构

122B `.log` 文件不是普通文本日志，而是 LevelDB WAL。其逻辑内容可解析为：

```
WriteBatch:
  sequence: 8 bytes
  count: 1
  operation: put
  key: mVillages
```

```
value: 64#...
```

其中 `mVillages` 是明文键名，`64#...` 是多玩加密或封装后的值，不是 MCPE 原始村庄数据。

6.2 为什么运行一次游戏后 log 变成 0B

MCPE 打开地图时会打开 LevelDB。LevelDB 会重放 `.log` 中的 WAL，把 `mVillages=64#...` 写入当前数据库状态。随后 LevelDB 可能压缩、轮转或截断该 `.log`，因此用户观察到：

- ? 不同地图里的 log 固定为 122B。
- ? 放入游戏存档目录运行一次后，该 log 变成 0B。
- ? 对应加密值已经合并进 `.ldb` 或新的 LevelDB manifest 状态。

6.3 为什么直接打开会导致地图损坏或崩溃

如果 MCPE 没有多玩悬浮窗插件 hook，它无法理解 `mVillages` 中的 `64#...` 加密值。该异常值进入 LevelDB 后，可能导致 MCPE 在读取村庄数据、世界数据或 LevelDB 状态时失败。

此外，如果修复工具只删除 `.log` 而不修复已经合并进 `.ldb` 的值，地图仍可能损坏。反过来，如果修复 `.ldb` 后没有补齐 LevelDB 新生成的 `MANIFEST-xxxxx`，也会导致数据库无法打开。

6.4 当前修复策略

当前 Android 工具修复策略如下：

1. ZIP 或目录中遇到可疑 122B `.log` 时，默认跳过输出，不写 0B 文件。
2. 将 `db/` 复制到 app cache。
3. 使用原 APK 兼容的 `libleveldbjni.so` 打开 LevelDB。
4. 读取键 `mVillages`。
5. 如果值形如 `数字#密文`，调用 LevelDB delete 删除该键。
6. 再次读取验证是否删除成功。
7. 将修复后的完整 DB 文件集复制回输出。
8. 对 ZIP 模式，额外补写所有新生成的 LevelDB 文件，避免 `CURRENT` 指向不存在的 manifest。

默认输出模式不保留调试副产物，适合导入 MCPE。可选开启 `Keep Repair Artifacts` 时，工具会保留：

- ? `*.dw_injected_wal.bak`
- ? `mVillages.dw_merged_value.bak`
- ? `*.dw_mVillages_slice.bin`

这些文件用于研究原始注入内容，不适合默认放入可导入地图中。

7. ZIP 文件名编码与目录结构问题

多玩历史资源可能使用 GBK 文件名或非 UTF-8 ZIP 标志。工具需要兼容：

? UTF-8 ZIP 名称。

? GBK 回退解码。

曾观察到手机上出现乱码目录 `鎴樼泩棕庚簩`，对应原名 `战地风云`。这说明 ZIP 名称编码对导入结果有实际影响。

另一个重要问题是资源包目录层级。错误输出可能形成：

```
resource_packs/resource_packs/...
behavior_packs/behavior_packs/...
```

正常应保持世界包结构与 `world_resource_packs.json`、`world_behavior_packs.json` 中引用的 UUID、版本匹配。结构错误可能导致资源包加载失败或 MCPE 崩溃。

8. 当前工具实现摘要

Android 解密工具工程位于：

`H:/dw-decryptor-android`

核心类：

? `app/src/main/java/com/example/dwdecryptor/DwDecryptor.java`

? `app/src/main/java/com/example/dwdecryptor/MainActivity.java`

? `app/src/main/java/com/mcbox/pesdk/util/LauncherMiscUtil.java`

native 依赖：

? `liblauncherutil.so`

? `libleveldbjni.so`

? `libcrypto.so`

? `libssl.so`

? `libgnustl_shared.so`

当前 UI 功能：

? 选择资源目录。

? 保存日志。

? `Repair Map DB`：默认开启，修复地图 LevelDB。

? `Keep Repair Artifacts`：默认关闭；开启后保留研究用修复副产物。

9. 验证结果

已完成的验证包括：

? modpkg 可解密。

? 地图 ZIP 可解密。

? 材质包可解密并加载。

? 行为包可解密并加载。

? 122B WAL 不再污染输出地图。

? 已合并进 `.ldb` 的 `mVillages` 加密值可逻辑删除。

? 修复后的 LevelDB `CURRENT` 指向的 manifest 能被完整写回 ZIP。

? 原版 MCPE 导入并启动地图不再崩溃。

? 多玩导入同一批新解密地图不再崩溃。

因此，可以认为当前样本集中的多玩地图资源已经完成可用性恢复。但这不等于所有历史版本、所有资源类型都百分百覆盖。若遇到新样本，仍应检查解密日志中的：

? `FAIL`

? `residual name`

? `residual magic`

? `residual db marker`

? `residual db current missing`

? `leveldb delete still present`

10. 后续研究方向

1. 对 `liblauncherutil.so` 内部算法做静态逆向，确认 `数字#密文` 的具体编码、分组和对称算法。
2. 对 `lp_v*.dat` 或释放出的插件 APK 做更深入分析，确认悬浮窗 hook 的完整调用链。
3. 增加批量校验器，自动检查输出 ZIP 的 LevelDB manifest、资源包 manifest、世界包引用和残留加密标志。
4. 将调试 artifact 输出到独立研究目录，而不是嵌入地图 ZIP，以进一步降低误导入风险。
5. 针对旧版本 MCPE 专用地图增加版本提示和兼容性分类。

11. 附录：关键特征速查

特征	含义
`19311931EE`	多玩头部封装加密 magic
`.level.key`	地图内容 key 元数据
`.user.key`	用户绑定与 `.level.key` MD5 校验数据
`.blur`	材质包、行为包或地图内包 key
`.texture.key`	皮肤、材质相关 key
`-mc` / `.json-mc`	加密 JSON 文件后缀
`数字#密文`	native 解密函数可处理的文本密文格式
`@`	多段密文分隔符
122B `.log`	LevelDB WAL，注入 `mVillages=64#...`

特征	含义
`mVillages`	MCPE LevelDB 键名，被多玩用于注入加密村庄数据
`CURRENT`	LevelDB 当前 manifest 指针
`MANIFEST-xxxxx`	LevelDB 元数据文件，缺失会导致 DB 损坏